9. Bölüm

Göstericiler ve Referanslar

9.1 Gösterici nedir? Niçin İhtiyaç Duyarız?

Şu ana kadar gördüğümüz `char`, `int`, `long`, `float`, `double` tipli değişkenler ve bunlara ait diziler, değer tipler (value type) olarak adlandırılır. Çünkü kimliklendirilen değişken, değeri doğrudan ifade eder. Değer tiplerin yanı sıra değişkenlerin adreslerini tutan değişkenlere de ihtiyaç duyarız. İşte adres tutan bu değişkenlere **gösterici tipler** (pointer type) adını veririz. Göstericileri işaret ettikleri veri tipine (data type) göre tanımlarız. Bu nedenle göstericiler, **türetilmiş tipler**(derived type) olarak adlandırılır.

Gösterici tipler, gösterdiği yerdeki verileri değiştirmek için de kullanılır. **Gösterici tipler işaret ettiği verinin tipine göre tanımlanırlar. Bütün gösterici tipler bellekte aynı miktarda yer kaplar. Çünkü hepsi adres tutar**. Bütün bu tanımlamaların ışığında göstericileri, vatandaşların ikametlerini gösteren adres numaraları olarak düşünebiliriz.

Gösterici tiplere ihtiyaç duyulmasının sebebi hız ve esnekliktir. Genel olarak birkaç madde ile sıralayacak olursak;

- Belleğin istenilen bölgesine erişim sağlamak için göstericiler kullanılır. Günümüz modern dillerinde **referans tipler** (referans type) kullanılır. Göstericilerden farklı olarak referans tipler, gösterilen adreste bir verinin/nesnenin bulunduğu garanti eder. Göstericiler ise her bellek bölgesine erişebilir.
- Bazen çalışma anında (run-time) yeni bellek bölgelerine ihtiyaç duyarız bu durumda göstericiler gereklidir. **Dinamik veri yapıları** (dynamic data structure) göstericiler yardımıyla oluşturulup yönetilir.
- Fonksiyonlara parametre geçirilirken argüman olarak kullanılan yerel değişkenlerin kopyaları kullanılır. Bu durum Şekil 7.3'de açıklanmıştı. Fonksiyondan geri dönüldüğünde ise yerel değişkenler eski haline yığın bellekten geri çekilerek çevrilir. Fonksiyonun yerel değişken ya da argümanlardan birini değiştirmesi istendiğinde fonksiyona argüman olarak değişkenin adresi verilir. Böylece fonksiyona parametre olarak geçirilen yerel değişkenler göstericiler üzerinden fonksiyon içinde değiştirilebilir hale gelir.

Göstericilere olan ihtiyacı bir başka örnekle de açıklayabiliriz; Bir ormandaki ağaçları boylarına göre sıralamaya kalkarsak her birini yer değiştirme yöntemiyle sıralamak oldukça külfetli ve zaman alan bir işlemdir. Halbuki ağaçlara numara vererek ve bu numaraların karşısına uzunluklarını yazacağımız bir liste oluşturulduğunda sadece numaraları sıralamak oldukça kolay ve zahmetsiz bir işlemdir. İşte buradaki numaraları karşılık gelen değişkenler göstericilerdir.

9.2 Gösterici Tanımlama

Göstericiler, işaret ettiği verinin tipine göre tanımlanırlar. Gösterici tanımlanırken veri tipi izleyen **başvuru kaldırma işleci** (dereference operator) (`*`) kullanılır. Başvuru kaldırma deyimi, gösterici tarafından tutulan bellek adresinde saklanan değere erişmeyi ifade eder.

```
//GÖSTERİCİ TANIMLAMA:
veritipi *gostericiKimligi1;
veritipi* gostericiKimligi2;

//AYNI VERİ TİPİNDE TEK SATIRDA BİRDEN ÇOK GÖSTERİCİ TANIMLAMA:
veritipi* birinciGösterici, ikinci gösterici;

//AYNI VERİ TİPİNDE TEK SATIRDA HEM DEĞİŞKEN HEM DE GÖSTERİCİ TANIMLAMA:
veritipi değişkenKimliği, *göstericiKimliği3;
```

Göstericinin işaret ettiği değişkene değer atama aşağıdaki şekilde yapılır. Burada göstericinin veri tipi ile atanan değer aynı veri tipinde olmalıdır;

```
gostericikimligi = &degiskenkimliği;
```

Bir değişkenin adresinin **adres işleci** ile elde edilebileceği 4.9.2 başlığında anlatılmıştı. Aşağıda göstericilere ilişkin kod örnekleri verilmiştir;

```
char *ptrToChar;  /*göstericinin işaret ettiği adreste "char" tipinde veri olan "ptrToChar"
 ↪  kimlikli göstericisi tanımlandı.*/

int i=10;
int *ptrToInt=&i; /*göstericinin işaret ettiği adreste "int" tipinde veri olan "ptrToInt"
 ↪  kimlikli göstericisi tanımlandı. Ancak göstericinin işeret etteği adres olarak i değişkeninin
 ↪  adresi atanıyor.*/

*ptrToInt =20; /* "ptrToInt" göstericisinin işaret ettiği adresteki tam sayı değeri 20 yaptık.
 ↪  "ptrToInt" göstericisi, i değişkeninin adresini işaret ettiği i değişkeninin değeri de 20
 ↪  olmuştur */
```

Veri tiplerinin işlendiği 4.3 başlığında, **hiçbir değeri olmayan ve bellekte yer kaplamayan** `void` veri tipi anlatılmıştı. Veri tipi `void` olan `void*` göstericisine **jenerik gösterici** (generic pointer) adı verilir.

Bazı durumlarda göstericilerin veya gösterdiği değerlerin sabit olması gerekebilir. Bunun için üç durum ortaya çıkabilir;

1. **İşaret edilen değerin sabit olması durumu**: Bu durumda gösterici tanımlaması aşağıdaki gibi yapılır;
```
int main() {
    int i=10;
    const int *ptrToi=&i; // değerin sabit olması durumunda gösterici tanımı
    *ptrToi=20; /* HATA: error: assignment of read-only location '*ptrToi'
    pi göstericisinin işaret ettiği adresteki tam sayı değeri 20 yapılamaz. */
}
```
2. **Göstericinin işaret ettiği adresin sabit olması durumu**: Kısaca göstericinin sabit olması durumunda tanımlama aşağıdaki gibi yapılır;
```
int main() {
    int i=10;
    int *const ptrToi=&i; // göstericinin sabit olması durumunda gösterici tanımı
    /* Bu tür tanımlamada bir adres ilk değer (initialize) olarak mutlaka verilmelidir. */
    int j=30;
    *ptrToi=20; // pi göstericisinin işeret ettiği adresteki tam sayı değeri 20 olur.
    ptrToi =&j; /*HATA:error: assignment of read-only variable 'ptrToi'
    pi göstericisinin işeret ettiği adres değiştirilemez!*/
}
```
3. **Hem göstericinin hem de değerin sabit olması durumu**:
```
int main() {
    int j=10, k=100;
    const int * const ptr=&j; /* Hem gösterici, hem de gösterilen veri sabit */
    /* Bu tür tanımlamada da bir adres ilk değer (initialize) olarak mutlaka verilmelidir.
     ↪  */
    *ptr=20; /* HATA: error: assignment of read-only location '*(const int *)ptr'*/
    ptr=&k; /*HATA: error: assignment of read-only variable 'ptr' */
}
```

Gösterici tipe atamaya uygun kesin bir adresiniz yoksa, `nullptr` **boş göstericisini** (null pointer) atamak her zaman iyi bir uygulamadır. İlk değeri olmayan gösterici tanımlamak yerine, ilk değeri `nullptr` olan göstericiler tanımlanmak doğru bir yöntemdir. Aşağıdaki gibi kullanılır;

```
veritipi *gostericikimligi = nullptr; //tanımlarken
gostericikimligi = nullptr; //tanımlı bir göstericiye nullptr ataması
```

Aşağıda boş gösterici kullanımına ilişkin örnek bir uygulama verilmiştir;

```
#include <iostream>
int main(){
    int *ptr = nullptr;
    std::cout << "ptr göstericisinin işaret ettiği adres: " <<  ptr << std::endl;
    int agirlik=80;
    ptr=&agirlik;
    std::cout << "ptr göstericisinin işaret ettiği adres: " << ptr << " ve değeri: " << *ptr
     ↪  <<std::endl;
}
/* Program Çıktısı:
```

```
ptr göstericisinin işaret ettiği adres: 0
ptr göstericisinin işaret ettiği adres: 0x7ffda3348764 ve değeri: 80
*/
```

Bir göstericinin işaret ettiği yerde bir başka gösterici olabilir. Buna **çifte gösterici** (double pointer) adı verilir. Aşağıda buna ilişkin bir örnek verilmiştir;

```
#include <iostream>
int main(){
    int var = 10;
    int *intPtr = &var;
    int **ptrToPtr = &intPtr;  //double pointer
    std::cout << "var:" << var << " &var:"<< &var<< std::endl;
    std::cout << "inttptr:"<< intPtr <<" *inttptr:" << &intPtr <<std::endl;
    std::cout << "var:"<< var <<" *intPtr:" << *intPtr << std::endl;
    std::cout << "ptrToPtr:" << ptrToPtr <<" &ptrtptr:" << &ptrToPtr << std::endl;
    std::cout << "&intPtr:" << &intPtr << " *ptrToPtr:" << *ptrToPtr << std::endl;
    std::cout << "var:" << var <<" *intPtr:" << *intPtr <<" **ptrToPtr:" << **ptrToPtr <<
     ↪  std::endl;
}
/*Program Çıktısı:
var:10 &var:0x7ffc7a8533ac
inttptr:0x7ffc7a8533ac *inttptr:0x7ffc7a8533a0
var:10 *intPtr:10
ptrToPtr:0x7ffc7a8533a0 &ptrtptr:0x7ffc7a853398
&intPtr:0x7ffc7a8533a0 *ptrToPtr:0x7ffc7a8533ac
var:10 *intPtr:10 **ptrToPtr:10
*/
```

Bir tam sayı bellekte 4 bayt yer işgal eder. Bu baytların bellekte nasıl yerleştiğini gösteren program verilmiştir.

```
#include <iostream>
#include <iomanip> // std::hex, std::setw ve std::setfill için
int main() {
    int i = 0x10203040;
    unsigned char* p = reinterpret_cast<unsigned char*>(&i); // C++'da daha gösterici dönüşümleri
     ↪  için güvenli olan static_cast veya reinterpret_cast tercih edilir. Burada i tam sayısının
     ↪  adresinde "unsigned char" işaret eden bir gösterici tanımladık.
    std::cout << "Sayı  :" << std::setw(10) << std::dec << i << "-" << std::hex << i << std::endl;
    // Baytları yazdırma (1. bayttan 4. bayta)
    for (int n = 0; n < 4; ++n) {
        std::cout << n + 1 << ".bayt:" << std::setw(10) << std::dec
            << static_cast<int>(*(p + n)) // "unsigned char" sayısal değer için int'e cast edilir
            << "-" << std::hex << std::setw(8 - (n * 2)) << std::setfill(' ')
            << static_cast<int>(*(p + n)) << std::endl;
    }
}
/*Program Çıktısı:
Sayı  : 270544960-10203040
1.bayt:        64-      40
2.bayt:        48-    30
3.bayt:        32-  20
4.bayt:        16-10
*/
```

9.3 Dinamik Değişken Kullanımı

Şu ana kadar nesnelerin imal edildiği bellek bölgesi, yığın bellek bölgesidir (stack segment). Değişkenleri çalışma anında (run-time), öbek bellekte (heap segment) de imal edebiliriz. Değişkenleri dinamik başlatma aşağıdaki şekilde yapılır;

```
//Bellekte tek bir değişken için bellek tahsisi:
veri-tipi* göstericisi = new veri-tipi;
//Bellekte tek bir değişken için bellek tahsisi:
```

```
//Ancak veri tipinde bir değişmez ile başlatma yapılmak istenirse:
veri-tipi* göstericisi = new veri-tipi(ilk-değer);
//boyut kadar elemanı olan dizi değişkeni için bellek tahsisi:
veri-tipi* dizi-göstericisi = new veri-tipi[boyut];
```

Dinamik olarak **new işleci** (new operator) ile imal edilecek değişkene öbek bellekte (heap segment) yer tahsis edilir (memory allocation). Ardından tahsis edilen belleği işaret eden gösterici geri döner. Diziler için dizinin ilk elemanını işaret eden gösterici geri döner. Göstericin işaret ettiği adresteki veriye **başvuru kaldırma işleci** (dereference operator) (`->`) ile erişilir.

Dinamik olarak imal edilmiş nesne kullanıldıktan sonra **delete işleci** (delete operator) ile bellekten kaldırılabilir (deallocate memory). Aşağıda buna ilişkin bir örnek verilmiştir;

```
#include <iostream>
int main() {
    // 1. DİNAMİK TEKİL DEĞİŞKEN (Geleneksel Yöntem)
    double* dPtr = new double; // tek değişkene bellek tahsis ediliyor.
    *dPtr = 3.14;
    std::cout << "Double Deger: " << *dPtr << " | Adres: " << dPtr << std::endl;
    delete dPtr; // tek değişken için tahsis edilen bellek serbest bırakılıyor.
    double* dPtr2 = new double(3.1415); // tek değişkene bellek tahsis ediliyor.
    std::cout << "Double Deger: " << *dPtr << " | Adres: " << dPtr << std::endl;
    delete dPtr2; // tek değişken için tahsis edilen bellek serbest bırakılıyor.
    // 2. DİNAMİK DİZİ (Geleneksel Yöntem)
    int boyut = 5;
    double* diziPtr = new double[boyut]; // 5 elemanlık yer açıldı
    diziPtr[0] = 10.5;
    diziPtr[1] = 20.7;
    diziPtr[2] = 30.9;
    //Atanmayan dizi elemanlarına dikkat ediniz!
    std::cout << "--- Dinamik Dizi Elemanlari ---" << std::endl;
    for (int i = 0; i < boyut; i++) {
        std::cout << i << ". eleman: " << diziPtr[i] << " | Adres: " << &diziPtr[i] << std::endl;
    }
    delete[] diziPtr; // KRİTİK: Dizi serbest bırakılırken mutlaka köşeli parantez kullanılmalı!
    // 3. MODERN YAKLAŞIM: Akıllı Göstericiler (Smart Pointer - unique_ptr)
    // 'delete' yazmanıza gerek kalmaz, kapsam bitince kendi silinir. İleride Anlatılacaktır.
    std::unique_ptr<double[]> akilliDizi = std::make_unique<double[]>(2);
    akilliDizi[0] = 100.1;
    akilliDizi[1] = 200.2;
    std::cout << "Akilli Isaretci ile Dizi: " << akilliDizi[0] << ", " << akilliDizi[1] <<
    ↪ std::endl;
}
/*
Double Deger: 3.14 | Adres: 0x21ad92b0
Double Deger: 3.1415 | Adres: 0x21ad92b0
--- Dinamik Dizi Elemanlari ---
0. eleman: 10.5 | Adres: 0x21ad96e0
1. eleman: 20.7 | Adres: 0x21ad96e8
2. eleman: 30.9 | Adres: 0x21ad96f0
3. eleman: 0 | Adres: 0x21ad96f8
4. eleman: 0 | Adres: 0x21ad9700
Akilli Isaretci ile Dizi: 100.1, 200.2
*/
```

9.4 Gösterici Aritmetiği

Göstericilerde toplama ve çıkarma tam sayılardan farklı şekilde çalışır. **Bir gösterici artırıldığında veya azaltıldığında, işaret ettiği adres, işaret ettiği veri tipinin bellek boyutu kadar artırılır veya azaltılır.** Bunun nedeni bilgisayarların tasarımından ötürü işlemciye bağlı belleklerde her bir bayt ayrı adreslenmesidir.

Bir `char*` göstericisi, işaret ettiği yerdeki `char` verisinden bir sonraki `char` verisini göstermesi istendiğinde **gösterici adresi 1 artar**. Çünkü `char` veri tipi bellekte 1 bayt yer kaplar. Aşağıdaki program çıktısındaki adres değişimlerini inceleyiniz;

```
#include <iostream>
const int BOYUT = 5;
void printDizi(const char* d) {
    std::cout << "DİZİ:{";
        for (int i = 0; i < BOYUT; i++)
        std::cout << "'" << d[i] << "'" << (i < BOYUT - 1 ? "," : "");
        std::cout << "}" << std::endl;
}
int main() {
    char dizi[BOYUT] = {'I', 'L', 'H', 'A', 'N'};
    // Dizinin ilk elemanını işaret eden gösterici
    char* ptrToChar = &dizi[0];
    // C++'da char* adresini yazdırmak için void*'e cast etmek şarttır
    std::cout << "İşaret Edilen Adres: " << static_cast<void*>(ptrToChar) << std::endl;
    *ptrToChar = 'X';
    printDizi(dizi);
    // Göstericiyi bir sonraki bayta (karaktere) ilerlet
    ptrToChar++;
    std::cout << "İşaret Edilen Adres: " << static_cast<void*>(ptrToChar) << std::endl;
    *ptrToChar = 'Y';
    printDizi(dizi);
    // Göstericiyi 3 adım ileri taşı (L'den N'ye)
    ptrToChar += 3;
    std::cout << "İşaret Edilen Adres: " << static_cast<void*>(ptrToChar) << std::endl;
    *ptrToChar = 'Z';
    printDizi(dizi);
}
/*Program Çıktısı:
İşaret Edilen Adres: 0x7ffe0bfe4523
DİZİ:{'X','L','H','A','N'}
İşaret Edilen Adres: 0x7ffe0bfe4524
DİZİ:{'X','Y','H','A','N'}
İşaret Edilen Adres: 0x7ffe0bfe4527
DİZİ:{'X','Y','H','A','Z'}
*/
```

Benzer şekilde `int*` göstericisi, işaret ettiği yerdeki `int` verisinden bir sonraki `int` verisini göstermesi istendiğinde **gösterici adresi 4 artar**. Çünkü `int` veri tipi bellekte 4 bayt yer kaplar. Aşağıdaki program çıktısındaki adres değişimlerini inceleyiniz;

```
#include <iostream>
const int BOYUT = 5;
void printDizi(const int* d) {
    std::cout << "DİZİ:{";
        for (int i = 0; i < BOYUT; i++)
        std::cout << "'" << d[i] << "'" << (i < BOYUT - 1 ? "," : "");
        std::cout << "}" << std::endl;
}
int main() {
    int dizi[BOYUT] = {10, 20, 30, 40, 50};
    // Dizinin ilk elemanını işaret eden gösterici
    int* ptrToInt = &dizi[0];
    // C++'da int* adresini yazdırmak için void*'e cast etmek şarttır
    std::cout << "İşaret Edilen Adres: " << static_cast<void*>(ptrToInt) << std::endl;
    *ptrToInt = -1;
    printDizi(dizi);
    // Göstericiyi bir sonraki bayta (karaktere) ilerlet
    ptrToInt++;
    std::cout << "İşaret Edilen Adres: " << static_cast<void*>(ptrToInt) << std::endl;
    *ptrToInt = -2;
    printDizi(dizi);
    // Göstericiyi 3 adım ileri taşı
    ptrToInt += 3;
```

```
    std::cout << "İşaret Edilen Adres: " << static_cast<void*>(ptrToInt) << std::endl;
    *ptrToInt = -3;
    printDizi(dizi);
}
/*Program Çıktısı:
İşaret Edilen Adres: 0x7ffd26121c90
DİZİ:{'-1','20','30','40','50'}
İşaret Edilen Adres: 0x7ffd26121c94
DİZİ:{'-1','-2','30','40','50'}
İşaret Edilen Adres: 0x7ffd26121ca0
DİZİ:{'-1','-2','30','40','-3'}
*/
```

Bunun gibi;

- `short*` göstericisi, işaret ettiği yerdeki `short` verisinden bir sonraki `short` verisini göstermesi istendiğinde gösterici adresi 2 artar. Çünkü `short` veri tipi bellekte 2 bayt yer kaplar.
- `long*` göstericisi, işaret ettiği yerdeki `long` verisinden bir sonraki `long` verisini göstermesi istendiğinde gösterici adresi 4 artar. Çünkü `long` veri tipi bellekte 4 bayt yer kaplar;
- `float*` göstericisi, işaret ettiği yerdeki `float` verisinden bir sonraki `float` verisini göstermesi istendiğinde gösterici adresi 4 artar. Çünkü `float` veri tipi bellekte 4 bayt yer kaplar.
- `double*` göstericisi, işaret ettiği yerdeki `double` verisinden bir sonraki `double` verisini göstermesi istendiğinde gösterici adresi 8 artar. Çünkü `double` veri tipi bellekte 8 bayt yer kaplar;

Ayrıca göstericinin artırılması yanında eksiltilmesi de mümkündür. Bu durumda `-`, `-=` ve `--` işleçleri kullanılır.

- Görüldüğü üzere **göstericilerin üzerinde işleçler (operator) işlem yaparken göstericinin işaret ettiği verinin tipine göre farklı işlem yaparlar.**
- Gösterici aritmetiğinde (`++`, `--`, `+`, `-`, `+=` ve `-=`) ile (`<`, `>` ve `==`) çalışır. **Diğer işleçler çalışmaz.**

9.5 Gösterici Dizileri

Göstericiler de dizi olarak tanımlanabilir. Aşağıda birbirinden bağımsız tanımlanmış yerel değişken adreslerinden bir gösterici dizisi tanımlanmıştır. Bu dizi üzerinden değişkenler daha kolay işlenebilir;

```
#include <iostream>
#include <vector>
int main() {
    int i = 10, j = 20;
    float f = 1.0; // Kodda doğrudan kullanılmıyor ama yerini koruduk
    int k = 30, l = 40;
    // C++'da ham dizi yerine std::array de kullanılabilir,
    // ancak mantığı korumak için gösterici dizisi (pointer array) tanımlıyoruz:
    int* ptr[4];
    // Göstericilere adres atamaları
    ptr[0] = &i; // i'nin adresi
    ptr[1] = &j; // j'nin adresi
    ptr[2] = &l; // l'nin adresi
    ptr[3] = &k; // k'nin adresi
    // ptr[2] (l değişkeni) üzerinden değer atama
    *ptr[2] = -1;
    // Değerlere ulaşma ve yazdırma
    for (int indis = 0; indis < 4; indis++) {
        std::cout << "ptr[" << indis << "] ile gösterilen değer: " << *ptr[indis] << std::endl;
    }
}
/* Program Çıktısı:
ptr[0] ile gösterilen değer: 10
ptr[1] ile gösterilen değer: 20
ptr[2] ile gösterilen değer: -1
ptr[3] ile gösterilen değer: 30
*/
```

9.6 Referanslar

Referanslar (reference), bellekte halihazırda var olan bir değişkenin **takma adı** (alias) gibidir. Bir değişken gibi kendisine yeni bir bellek alanı tahsis edilmez, sadece mevcut bir değişkene ilişkin bellek bölgesine ikinci bir isim verir. Referansın adresi ile bağlandığı değişkenin adresi tamamen aynıdır. Bir değişken, bildiriminde kimlik önüne (&) konularak referans olarak bildirilebilir.

```
veri-tipi varolan-bir-değişken;
veri-tipi &referans-kimliği = varolan-bir-değişken;
veri-tipi& referans-kimliği = varolan-bir-değişken;
```

Aşağıda buna ilişkin örnek bir program verilmiştir;

```
#include <iostream>
using namespace std;
int main(){
    int x = 10;
    int &ref = x; // ref ile var olan x değişkenine referans tanımlanmıştır.
    ref = 20; // x değişkeninin yeni değeri 20 olmuştur.
    cout << "x = " << x << endl; // x = 20
    x = 30; // x değişkeninin yeni değeri 30 olmuştur
    cout << "ref = " << ref << endl; // ref = 30
}
```

Özellikle fonksiyonlara büyük veriler (örneğin binlerce elemanlı bir dizi veya büyük bir nesne) gönderirken kopyalama maliyetinden kurtulmak için kullanılır. Değerle gönderirseniz her seferinde kopyası alınır bu da icra süresini uzatır yani yavaştır, referansla gönderirseniz orijinal veri üzerinden işlem yapılır bu da icra süresini çok kısaltır yani çok hızlıdır.

Referanslar ile göstericileri karşılaştırdığımızda;

- Göstericiler herhangi bir zamanda boş gösterici (NULL Pointer) olabildiği gibi başka bir nesneyi işaret edilebilir. Bir referans tanımlandığı anda ise mevcut bir değişken ile ilk değer verilmelidir. Göstericiler tanımlandıktan sonra herhangi bir zamanda değeri değişebilir ya da gösterdiği değişken değiştirilebilir.
- Hiçbir değişkeni işaret etmeyen boş referansınız (NULL Reference) olamaz! Her zaman bir referansın meşru bir değişkene bağlı olduğunu varsayabilmelisiniz.
- Bir referans imal edilen bir nesneyi işaret ediyorsa, başka bir nesneyi göstermek üzere referansı değiştirilemez!

Göstericiler yerine referanslar kullanmak kodumuzun okunmasını ve bakımını daha kolay hale getirilebilir. Bir C++ fonksiyonu, bir gösterici döndürdüğü gibi bir referansı da döndürebilir. Bir fonksiyon bir referans döndürdüğünde, dönüş değerine örtük bir gösterici döndürür. Böylece atama ifadesinin sol tarafında bir fonksiyon kullanılabilir hale gelir.

```
#include <iostream>
#include <ctime>
float dizi[5] = {10.0, 20.0, 30.0, 40.0, 50.0};
float& indistekiDeger( int indis ) { //dizinin bir elemanının referansını döndüren bir fonksiyon
    return dizi[indis];
}
int main () {
    std::cout << "Değiştirmeden Önce Dizi" << std::endl;
    std::cout << "{";
    for ( int i = 0; i < 5; i++ )
        std::cout << dizi[i] << ",";
    std::cout << "}" << std::endl;
    //Referanslar üzerinden değerler değiştiriliyor...
    indistekiDeger(1) = 100.0;
    indistekiDeger(3) = 200.0;
    std::cout << "Değiştirildikten Sonra Dizi" << std::endl;
    std::cout << "{";
    for ( int i = 0; i < 5; i++ )
        std::cout << dizi[i] << ",";
    std::cout << "}" << std::endl;
```

```
}
/* Program Çıktısı:
Değiştirmeden Önce Dizi
{10,20,30,40,50,}
Değiştirildikten Sonra Dizi
{10,100,30,200,50,}
*/
```

Bir fonksiyondan bir referans döndürürken, referans alınan nesnenin kapsam (scope) dışına çıkmamasına dikkat edilmesi gerekir;

```
int& fonksiyon() {
    int q;
    // return q; /* Derleme hatası olur. Çünkü fonksiyon dışına çıkıldığında bu değişken bellekte
    ↪ yığın bellekte yoktur.*/
    static int x;
    return x; /* x değişkeni tüm program süresince hayatta olduğundan bu kod güvenlidir. */
}
```

9.7 Referansla Çağırma

Fonksiyonlara parametre geçirilirken argüman olarak kullanılan yerel değişkenlerin kopyaları kullanılır. Bu durum Şekil 7.3'de açıklanmıştı. Fonksiyondan geri dönüldüğünde ise yerel değişkenler eski haline yığın bellekten geri çekilerek çevrilir. Buna ilişkin örnek aşağıda verilmiştir;

```
#include <iostream>
// Fonksiyon Bildirimi/Prototipi
void degistir(int, int);
int main() {
    int a = 10, b = 20;
    degistir(a, b);
    // Çağrı sonrası yerel değişkenler yığın bellekten çekildiği için değerler değişmez
    std::cout << "1- a:" << a << ", b:" << b << std::endl; // Çıktı: a:10, b:20
}
void degistir(int pX, int pY) {
    int z = pX;
    pX = pY;
    pY = z;
}
```

Fonksiyona geçirilen yerel değişken ya da argümanlardan birinin değiştirilmesi istendiğinde bu fonksiyona argüman olarak değişkenin adresi verilir. Böylece fonksiyona parametre olarak geçirilen yerel değişkenler dolaylı olarak göstericiler üzerinden fonksiyon içinde değiştirilebilir hale gelir. Fonksiyona geçirilecek argümanlar gösterici olarak tanımlanır. Çağrı ortamına geri dönüldüğünde yerel değişkenler de değişmiş olur. Bu işleme **göstericililer üzerinden referansla çağırma** (call by reference over pointer) adı verilir. Çünkü fonksiyona değişken adresleri değer olarak geçirilmiştir. Buna ilişkin örnek aşağıda verilmiştir;

```
#include <iostream>
// Fonksiyon prototipi: int* (gösterici) kabul eder
void degistir(int* pX, int* pY);
int main() {
    int a = 10, b = 20;
    // Değişkenlerin adreslerini (&) gönderiyoruz
    degistir(&a, &b);
    // Çıktı: a:20, b:10
    std::cout << "2- a:" << a << ", b:" << b << std::endl;
}
void degistir(int* pX, int* pY) {
    int z = *pX; // pX'in işaret ettiği adresteki değeri al
    *pX = *pY;   // pY'nin değerini pX'in adresine yaz
    *pY = z;     // Yedeklenen değeri pY'nin adresine yaz
}
```

Göstericiler üzerinden referansla çağırma yerine bir önceki 9.6 başlığındaki referanslar üzerinden fonksiyonu çağırmak daha güvenlidir. Buna referansla çağırma (call by reference) bu durumda yukarıdaki örnek

daha güvenli olarak aşağıda verilmiştir;

```
#include <iostream>
// Fonksiyon prototipi: int& (referans) kabul eder
void degistir(int& pX, int& pY);
int main() {
    int a = 10, b = 20;
    // Adres (&) veya yıldız (*) operatörüne gerek yok! Normal değişken gibi gönderiyoruz.
    degistir(a, b);
    // Çıktı: a:20, b:10
    std::cout << "3- Referans ile a:" << a << ", b:" << b << std::endl;
}
void degistir(int& pX, int& pY) {
    int z = pX; // pX artık main'deki 'a'nın ta kendisidir
    pX = pY;    // pY'den gelen değer 'a'ya yazılır
    pY = z;     // z'deki değer 'b'ye yazılır
}
```

9.8 Göstericilerin Dizileri İşaret Etmesi

Çoğu durumda, bir dizi yardımıyla gerçekleştirdiğimiz işleri gösterici ile de gerçekleştirilebiliriz. Dizi değişkeni, dizinin ilk öğesinin adresi işaret eder. Kısaca dizi değişkenleri gösterici gibi davranırlar.

```
#include <iostream>
int main() {
    int dizi[5] = {10, 20, 30, 40, 50};
    int* ptr = dizi; // Dizinin ilk elemanının adresini tutar
    std::cout << "Dizi elemanlarına gösterici ile erişim: " << std::endl;
    for (int indis = 0; indis < 5; ++indis)
        // *(ptr + indis) ifadesi pointer aritmetiği kullanarak değere ulaşır
        std::cout << "dizi[" << indis << "]: " << *(ptr + indis) << std::endl;
}
/* Program Çıktısı:
Dizi elemanlarına gösterici ile erişim:
dizi[0]: 10
dizi[1]: 20
dizi[2]: 30
dizi[3]: 40
dizi[4]: 50
*/
```

Fonksiyonlara dizileri işaret eden göstericileri parametre olarak verebiliriz. Dizi değişkeni, dizinin ilk öğesinin adresi işaret ettiğinden dolayı d**izi değişkenleri (array) parametre olarak kullanılırken adres işleci (address operator) kullanılmaz.** Bu durumda parametre olarak dizi elemanının veri tipinde bir göstericiyi parametre olarak tanımlarız.

```
float notlar[10];
float notlarOrtalamasi(float*); //prototype
//...
float ort=notlarOrtalamasi(notlar); // yada notlarOrtalamasi(&notlar[0]);
//...
float matris[4][3];
float defetminant(float**,int,int); //prototype
//...
float det=defetminant(matris,4,3);
//...
```

Aşağıda öğrencilerin notlarına ilişkin işlemler yapılan bir program verilmiştir.

```
#include <iostream>
#include <iomanip>
// C++'ta sabitler için constexpr veya const önerilir
constexpr int OGRENCISAYISI = 10;
float notlarOrtalamasi(const float*); // Ortalamada dizi değiştirilmediği için const eklendi
void notlariArtir(float*);
int main() {
```

```
    float notlar[OGRENCISAYISI] = { 55.0, 60.0, 70.0, 35.0, 30.0, 50.0, 65.0, 90.0, 95.0, 100.0 };
    float ortalama = notlarOrtalamasi(notlar);
    std::cout << "Notlar Ortalaması: " << std::fixed << std::setprecision(1) << ortalama <<
    ↪ std::endl;
    notlariArtir(notlar); // Dizi içeriği fonksiyon içinde değiştiriliyor
    for (int indis = 0; indis < OGRENCISAYISI; ++indis)
        std::cout << "Not[" << indis << "]=" << notlar[indis] << std::endl;
}
float notlarOrtalamasi(const float* pNotlar) {
    float top = 0.0f;
    for (int indis = 0; indis < OGRENCISAYISI; ++indis)
        top += pNotlar[indis];
    return top / OGRENCISAYISI;
}
void notlariArtir(float* pNotlar) {
    for (int indis = 0; indis < OGRENCISAYISI; ++indis)
        if (pNotlar[indis] <= 90.0f)
            pNotlar[indis] += 10.0f;
}
/* Programın Çıktısı:
Notlar Ortalaması: 65.0
Not[0]=65.0
Not[1]=70.0
Not[2]=80.0
Not[3]=45.0
Not[4]=40.0
Not[5]=60.0
Not[6]=75.0
Not[7]=100.0
Not[8]=95.0
Not[9]=100.0
*/
```

9.9 Fonksiyonlardan Bir Diziyi Geri Döndürme

C dilinde bir fonksiyonun geri dönüş değeri olarak tüm bir dizi verilemez! Ancak, bir dizinin göstericisini fonksiyondan geri dönüş değeri olarak döndürebilirsiniz. **Burada dikkat edilmesi gereken fonksiyondan çıkılınca dizinin bellekten kaldırılmamasıdır.** Çünkü yerel değişkenler yığın bellekte tutulduğundan fonksiyondan çıkılınca bellekten silinirler. Aşağıda bir tam sayı dizi göstericisini döndüren bir fonksiyon örnekleri verilmiştir;

```
int* tekBoyutluDiziDondur() {
 /*... */
}
int** ikiBoyutluDiziDondur() {
 /*... */
}
```

Yerel değişkenlerin yığın bellekte (stack segment) tutulduğu ve fonksiyondan geri dönünce fonksiyon yerelindeki değişkenlerin kaybolduğu 7.5.1 başlığında ve Şekil 7.3'de anlatılmıştı. Bu nedenle bir fonksiyon içinde tanımlanan bir dizinin göstericisi geri döndürülecek ise veri bellekte (data segment) yer almasını sağlayacak `static` tanımlamasıdır.

```
#include <iostream>
#include <iomanip> // setw için
#include <cstdlib> // rand ve srand için
#include <ctime>   // time için
constexpr int BOYUT = 10;
// Fonksiyon int* (gösterici) döndürüyor
int* rastgeleOgrenciNotlari() {
    // static dizi: Fonksiyon bitse bile bellekte kalmaya devam eder.
    static int notlar[BOYUT];
    for (int i = 0; i < BOYUT; ++i)
        notlar[i] = std::rand() % 101;
```

```
    return notlar; // Dizinin ilk elemanının adresini döndürür
}
void notlariYaz(const int* pNotlar, int pUzunluk) {
    std::cout << "Öğrenci Notları:" << std::endl;
    for (int i = 0; i < pUzunluk; ++i)
        std::cout << std::setw(4) << pNotlar[i];
    std::cout << std::endl;
}
int main() {
    // Rastgele sayı üretecini kur
    std::srand(static_cast<unsigned int>(std::time(nullptr)));
    // Fonksiyondan gelen adresi bir işaretçiye alıyoruz
    int* notlar = rastgeleOgrenciNotlari();
    notlariYaz(notlar, BOYUT);
}
/*Program Çıktısı:
Öğrenci Notları:
40  80  43  79  36  67  75  48  50  17
*/
```

C++ dilinde `static` dizi döndürmek tehlikeli olabilir (thread-safe değildir ve bellek yönetimini zorlaştırır). İleride göreceğimiz dinamik dizileri kullanarak (`std::vector`) nesneyi güvenli bir şekilde fonksiyondan dışarı taşıyabiliriz.

9.10 Fonksiyon Göstericisi

Geri çağırma (callback) fonksiyonları, özellikle **olay güdümlü programlamada** (event-driven programming) çok kullanışlıdır. Bir olay tetiklendiğinde, bu olaylara yanıt olarak ona eşlenen bir geri çağırma fonksiyonu çalıştırılır. Genellikle grafik ara yüzlü işletim sistemlerinde kullanıcı ara yüzünde bir düğmeye tıklanması gibi bir olay, önceden tanımlanmış bir dizi işlemi başlatabilir. Bu durum, geri çağırma işlevleriyle gerçekleştirilir.

Geri çağırma fonksiyonu, temel olarak, başka bir koda argüman olarak geçirilen ve belirli bir zamanda argümanı geri çağırması beklenen bir icra edilebilir koddur. Geri çağırma mekanizması fonksiyon göstericisine bağlıdır. **Fonksiyon göstericisi** (function pointer), bir fonksiyonun bellek adresini depolayan bir değişkendir. Aşağıda buna ilişkin basit bir örnek bulunmaktadır;

```
#include <stdio.h>
void merhaba() {
    printf("Merhaba!\n");
}
void callback(void (*ptr)()) {
    printf("Geri çağırma fonksiyonu çağrılacak!\n");
    (*ptr)();   // Geri çağırma fonksiyonu çağrılıyor
}
main() {
    void (*ptr)() = merhaba;
    callback(ptr);
}
```

9.11 Dinamik Hafıza Yönetimi

C++ dilinde bir fonksiyon bloğu içerisinde tanımlanan tüm değişkenler yığın bellek (stack segment) üzerinde yer kaplayacaktır. Program çalıştığında bir değişkene belleği çalışma anında (run-time) tahsis etmek için **öbek bellek** (heap segment) kullanılır. Buna **dinamik bellek tahsisi** adı verilir. Dinamik bellek tahsisi için `new` işlecini (new operator) kullanırız. Bu işleç, dinamik dizi için öbek bellekte yer tahsis edilip tahsis edilen bellek bölgesinin ilk baytının adresini göstericiye atar. Tahsis edilen bellek bölgesini serbest bırakmak için ise `delete` işleci (delete operator) kullanılır. Aşağıda bir `float` değişkene dinamik olarak yer tahsisi örneği verilmiştir;

```
#include <iostream>
int main(){
    float* ptrFloat=nullptr;
    ptrFloat=new float; // bir float değişkene heap segment üzerinde yer ayrılır ve adresi
     ↪ döndürülür.
    if (ptrFloat!=nullptr) {
```

```
        *ptrFloat=4.5;
        std::cout << *ptrFloat << std::endl;
    } else
        std::cout << "Bellek tahsisi yapılamadı!" << std::endl;
    delete ptrFloat; // tahsis edilen bellek bölgesi serbest bırakılır.
    ptrFloat=nullptr; /*Kod devam edecek ise bu talimat yazılmalıdır. */
}
```

C++ dilinde diziler de dinamik olarak göstericiler yardımıyla oluşturulur. Dinamik bellek tahsisi, çalışma zamanı sırasında bellek tahsis etmemize olanak tanır. Bir dizideki dinamik tahsis, özellikle bir dizinin boyutu derleme zamanında bilinmediğinde ve çalışma zamanı sırasında belirtilmesi gerektiğinde faydalıdır. Bir diziyi dinamik olarak tahsis etmek için;

- Tahsis edilen dizinin ilk elemanının adresini depolayacak bir gösterici tanımlanır.
- Sonra, belirli bir veri tipindeki bir diziyi barındıracak dizi için `new` işleci ile bellek alanını tahsis edilir.
- Bu tahsisi yaparken, kaç öğe içerebileceğini belirten dizinin boyutunu belirtilir. Bu belirtilen boyut, tahsis edilecek bellek miktarını belirler.

Dizi uzunluğu tam sayı olacak şekilde dinamik dizi aşağıdaki gibi tanımlanır;

```
veritipi* diziGöztericiKimliği= new veritipi[diziUzunluğu];
```

Aşağıda çalışma anında (run-time) dinamik olarak oluşturulan bir dizi örneği verilmiştir;

```
#include <iostream>
int main() {
    // Çalışma Anında Oluştutulacak Dizi Boyutu Soruluyor
    std::cout << "Dizinin Boyutunu Girin: ";
    int size;
    std::cin >> size;
    int* arr = new int[size]; // Diziye dinamik bellek (heap) tahsis ediliyor.
    if (!arr) { // if (arr!=nullptr) de yazılabilir
        for (int i = 0; i < size; i++) // Bir döngü ile dizi elemanlarına değer atanıyor
            arr[i] = i * 2;
        std::cout << "Dinamik Dizi Elemanları: " << std::endl;
        for (int i = 0; i < size; i++)
            std::cout << arr[i] << " ";
        std::cout << std::endl;
    }
    delete[] arr; // dizi bellekten kaldırılıyor. Tahsis edilen bellek serbest bırakılıyor.
}
/* Program Çıktısı:
Dizinin Boyutunu Girin: 5
Dinamik Dizi Elemanları:
0 2 4 6 8
*/
```

> C++'ta `new` ile ayrılan her bellek alanı, programcı tarafından mutlaka `delete` kullanarak elle serbest bırakılmalıdır. `delete` talimatı kullanılmazsa, program kapansa bile o bellek alanı işletim sistemi temizleyene kadar tahsis edilmiş kalabilir.

Çok boyutlu dizilere de dinamik olarak bellek tahsisi yapılabilir;

```
double** pvalue  = nullptr;   // Göstericiye ilk değer olarak nullptr veriliyor
pvalue  = new double [3][4];  // 3x4 buyutlu martis için yer tahsisi yapılıyor.
//...
delete[] pvalue; // tahsis edilen bellek serbest bırakılıyor
```

> C++'ta dinamik bellek yönetimi yaparken, manuel `new` ve `delete` kullanımı yerine Modern C++ (C++11/14/17/20) tekniklerini kullanmak, kodunuzu hem daha güvenli hale getirir hem de **bellek sızıntılarını** (memory leak) tamamen önler. İleride anlatılacaktır.

9.12 Metinler ve Göstericiler

C dilinden miras kalan ve C++'ta da hâlâ geçerli olan eski yöntem **metinler** (string), özünde birer **karakter dizisidir**. Modern `std::string` sınıfının aksine, bu yaklaşım çok daha düşük seviyelidir ve bellek yönetimi tamamen yazılımcının sorumluluğundadır.

§9.12.1 Karakter Dizisi

Eski yöntemde bir metin (string), `char` tipindeki verilerin yan yana dizildiği bir dizi (array) içinde saklanır. Ancak, bilgisayarın bu dizinin nerede bittiğini bilmesi gerekir. İşte bu noktada **boş karakter** (NULL Character) `'\0'` devreye girer. Boş karakter, değeri 0 olan özel bir karakterdir. Metnin bittiğini işaret eder. C stili tüm fonksiyonlar metni bu karaktere kadar okur. Bu karakter, metnin gerçek uzunluğuna +1 bayt eklenmesini gerektirir.

```
//Karakter Dizisi olarak dizgi tanımı:
char metin1[]={'C','L','a','n','g','\0'};
// metin1 dizgisi 6 karakterden oluşan dizi ile başlatılmıştır.
char metin2[]={'C','L','a','n','g', 0 };
// metin2 dizgisi 6 karakterden oluşan dizi ile başlatılmıştır.
char metin3[50]={'C','L','a','n','g', 0 };
// metin3 dizgileri 6 karakterden oluşmasına rağmen bellekte 30 bayt yer kaplar.
```

Metinleri oluşturan karakter dizilerine ilk değer vermek için çift tırnak karakteri kullanılır. Çift tırnak arasında metni oluşturan harflerin yazılmasıyla karakter dizisi (metin) değişmezleri (string literal) yazılır. Metni ifade eden karakter dizilerini uzun uzun dizi şeklinde başlatmak yerine metinler çift tırnak karakteri kullanılır;

```
char metin1[] = "CLang";
//Kısaca  6 karakterden oluşan dizgi tanımı
char metin2[40] = "Clang"
// metin3 dizgileri 6 karakterden oluşmasına rağmen bellekte 40 bayt yer kaplar.
```

Şimdiye kadar anlatılanlar ışığında dizgi tanımlama örneği aşağıda verilmiştir;

```
#include <cstdio> // C-stili I/O için C++'ta kullanılır
#include <iostream>
int main() {
    // YÖNTEM A: Boyut belirterek ve dizgi literal ile atama
    char dizi1[10] = "Merhaba";
    // "Merhaba" 7 karakterdir. '\0' için en az 8 bayt gerekir.
    // Kalan 2 bayt otomatik olarak '\0' ile doldurulur.
    // YÖNTEM B: Boyutu derleyiciye bırakma
    char dizi2[] = "Dunya";
    // Derleyici boyutu otomatik olarak 6 (5 harf + '\0') olarak ayarlar.
    // YÖNTEM C: Karakter karakter atama (Tavsiye edilmez, hata risklidir)
    char dizi3[6] = {'H', 'e', 'l', 'l', 'o', '\0'};
    // DİKKAT: '\0' unutulursa, fonksiyonlar bellekte rastgele okumaya devam eder!
    std::cout << "Dizi 1: " << dizi1 << std::endl;
    std::cout << "Dizi 2: " << dizi2 << std::endl;
    std::cout << "Dizi 3: " << dizi3 << std::endl;
}
```

§9.12.2 Gösterici ile Metinlerin İlişkisi

C++ dilinde bir dizinin adı, aslında o dizinin ilk elemanının bellek adresini gösteren **sabit bir göstericidir** (constant pointer). Çift tırnak içindeki metin değişmezleri (string literal) de bellekte bir yerde saklanır ve program o adresi kullanır. Bu durumda gösterici karakter dizisinin ilk elemanını gösterir;

```
char* metin3="CLang";
```

Buna ilişkin örnek program aşağıda verilmiştir;

```
#include <iostream>
int main() {
    char ad[] = "Ahmet"; // Değiştirilebilir bir karakter dizisi
    const char* ptr = "Ahmet"; // Bellekteki sabit bir metne tanımlanan gösterici
    // Adresin kendisini yazdırma
    std::cout << "Dizinin bellek adresi: " << (void*)ad << std::endl;
    std::cout << "Isaretcinin bellek adresi: " << (void*)ptr << std::endl;
```

```
    // İşaretçi aritmetiği
    std::cout << "ptr[0]: " << ptr[0] << std::endl; // 'A'
    std::cout << "ptr[1]: " << *(ptr + 1) << std::endl; // 'h'
    // DİKKAT: Göstericinin gösterdiği sabit metin değiştirilemez!
    // ptr[0] = 'B'; // HATA! Program çalışma anında çöker.
}
/*Program Çıktısı:
Dizinin bellek adresi: 0x7fff9f4397b2
Isaretcinin bellek adresi: 0x402004
ptr[0]: A
ptr[1]: h
*/
```

§9.12.3 Metinlerle İşlemler

C-stili karakter dizilerinde standart operatörler (=, +, ==) metin işlemleri için doğrudan çalışmaz. Bu işlemler için <cstring> başlık dosyasındaki fonksiyonlar kullanılır.

Atama işleci (= operator)dizinin içeriğini kopyalamaz, sadece adresi kopyalamaya çalışır (veya hata verir). İçeriği kopyalamak için strcpy() fonksiyonu kullanılır. **Bu kopyalamada hedef dizinin, kaynak diziyi ve boş karakteri alacak kadar büyük olduğundan emin olmalısınız.**

```
#include <iostream>
#include <cstring>
int main() {
    char kaynak[] = "Merhaba";
    char hedef[20]; // Yeterince büyük bir alan ayırdık
    // hedef = kaynak; // HATA! Dizilere doğrudan atama yapılamaz.
    strcpy(hedef, kaynak); // kaynak'ın içeriğini hedef'e kopyalar.
    std::cout << "Hedef: " << hedef << std::endl;
    std::cout << "Boyut: " << strlen(hedef) << " karakter" << std::endl;
}
```

Metin birleştirme (string concatenate) için toplama işleci (+ operator) yerine strcat() fonksiyonu kullanılır. Bu fonksiyon, bir metnin sonuna başka bir metni eklemek için kullanılır. Hedef dizinin, mevcut metin + eklenecek metin + boş karakteri alacak kadar büyük olduğundan emin olmalısınız. **Bu kontrol yapılmazsa tampon taşması (buffer overflow) hatası oluşur ve bu çok ciddi bir güvenlik açığıdır.**

```
#include <iostream>
#include <cstring>
int main() {
    char ad[30] = "Ahmet"; // Başlangıçta boş yer bıraktık
    char soyad[] = " Yilmaz";
    strcat(ad, soyad); // ad = ad + soyad gibi düşünün
    std::cout << "Tam Ad: " << ad << std::endl; //Tam Ad: Ahmet Yilmaz
}
```

Metinleri karşılaştırma (string compare) için strcmp() fonksiyonu kullanılır. eşit mi işleci (== operator) metinlerin içeriğini değil, bellekteki adreslerinin aynı olup olmadığını karşılaştırır. İçerik karşılaştırması için strcmp() kullanılır. Dönüş değeri 0 ise iki metin birbirine eşittir. Dönüş değeri sıfırdan farklı ise metinler farklıdır (alfabetik sıraya göre farkın yönünü gösterir). Üstelik sadece ASCII (American Standard Code for Information Interchange) kodları üzerinden karşılaştırma yapar.

```
#include <iostream>
#include <cstring>
int main() {
    char s1[] = "elma";
    char s2[] = "elma";
    char s3[] = "armut";
    // HATALI KULLANIM:
    if (s1 == s2) { // Bu muhtemelen 'yanlis' döner, çünkü adresler farklıdır.
        std::cout << "[==] Metinler esit (HATALI MANTIK)" << std::endl;
    }
    // DOGRU KULLANIM:
    if (strcmp(s1, s2) == 0)
    std::cout << "[strcmp] s1 ve s2 esit" << std::endl; //[strcmp] s1 ve s2 esit
```

```
    if (strcmp(s1, s3) != 0)
    std::cout << "[strcmp] s1 ve s3 farkli" << std::endl; //[strcmp] s1 ve s3 farkli
}
```

§9.12.4 Metinler İçin Tavsiye Edilen Yöntem

Gördüğünüz gibi eski yöntem çok fazla dikkat gerektirir ve bellek taşması hatalarına çok açıktır. Bu yüzden modern C++ kodlarında, bu düşük seviyeli detaylarla uğraşmamak ve daha güvenli, okunabilir kod yazmak için her zaman `std::string` sınıfını kullanmalısınız. İleride anlatılacaktır. C-stili metinleri sadece eski C kütüphaneleriyle uyumluluk gibi zorunlu durumlarda tercih etmelisiniz. Aşağıda modern kullanıma yönelik örnek kullanım verilmiştir.

```
#include <iostream>
#include <string> // string sınıfı için gerekli
int main() {
    std::string metin1;
    std::cout << "metin1:" << metin1 << std::endl; //metin1:
    metin1="İlhan";
    std::cout << "metin1:" << metin1 << std::endl; //metin1:İlhan
    std::string metin2=metin1;
    std::cout << "metin2:" << metin2 << std::endl; //metin2:İlhan
    metin2="İlhan";
    if (metin1==metin2)
        std::cout << "metin1 ile metin2 eşittir." << std::endl; //metin1 ile metin2 eşittir.
    metin2="Özkan";
    std::string metin3=metin1+" "+metin2;
    std::cout << "metin3:" << metin3 << std::endl; metin3:İlhan Özkan
}
```

9.13 Düşün ve Çöz

- **Düşün**: "Bir gösterici aslında sadece bir tam sayıdır (adres)" ifadesi doğru mudur? Gösterici tipi (`int*`, `char*` vb.) gösterici aritmetiğini nasıl etkiler?
- **Düşün**: `const int *p;` ile `int * const p;` tanımları arasındaki farkı bellek ve erişim yetkisi açısından açıklayınız.
- **Düşün**: Göstericiler ile diziler arasındaki adres-offset ilişkisini, dizi ismiyle gösterici aritmetiği yaparak açıklayınız.
- **Çöz**: Bir fonksiyon yazınız; bu fonksiyon iki tam sayının adresini parametre olarak alsın ve bu sayıların değerlerini bellekte yer değiştirsin.
- **Çöz**: Bir karakter dizgisini (string) gösterici aritmetiği kullanarak tersten ekrana yazdıran bir fonksiyon yazınız.
- **Çöz**: Bir fonksiyona parametre olarak başka bir fonksiyonun adresini gönderen (function pointer) basit bir matematiksel işlem seçici (topla/çıkar) tasarlayın.